Moodify — Implementation Plan

Version 3.0 (MVP — Expanded)

Sequenced build plan with full engineering documentation: requirements, risks, error handling, AI prompt strategy, API contracts, architecture, security, demo script, and testing checklist.

Document Map

This plan merges the original build-sequence plan with the additional engineering documentation recommended in review (success metrics, functional/non-functional requirements, risk analysis, error handling, state management, AI prompt strategy, API contracts, ER relationships, folder structure, coding standards, expanded security, demo scenario, future scope, and UI navigation map), plus project vision, architecture and ER diagrams, technology stack, demo data strategy, browser compatibility, and a complete screen inventory added in this revision. Sections are ordered so an AI coding agent (Cursor / Claude Code / Bolt / Replit) can read top-to-bottom before writing any code.

Project Vision

Elevator Pitch: Moodify turns a single selfie into an instant, judgment-free read on how you're feeling, paired with an AI-guided nudge toward one small action that helps.

Vision

A world where checking in on your emotional state is as fast and frictionless as checking the time — so people notice their own patterns before those patterns become a crisis.

Mission

Give people a private, always-available mirror for their emotional state, powered by computer vision and AI, that turns a 10-second scan into a concrete, personalized wellness suggestion.

Primary Goal

Ship a working end-to-end loop — scan → emotion → insight → suggestion → task → trend over time — that a judge or user can experience in under two minutes.

Why This Project Matters

Most wellness apps rely on the user to self-report accurately and consistently, which is exactly what people struggle to do when they're overwhelmed. Moodify lowers that barrier to almost zero: one photo replaces a mood journal entry, and the AI does the reflective work of noticing what the data means.

Expected Impact

For individual users: earlier self-awareness of mood dips and small, achievable actions instead of vague advice. For the hackathon: a technically ambitious but genuinely finishable demo that showcases computer vision, applied AI, and full-stack product thinking in one cohesive story.

0. Reading Order Before Writing Code

Feed the AI coding agent, in this order:

- This Implementation Plan (for sequencing, requirements, risks, and standards)
- PRD (for feature scope/intent)
- TRD (for stack/architecture decisions)
- Backend Schema Document (for exact DB/API shapes)
- App Flow Document (for routing/state logic)
- UI/UX Design Brief (for visual direction)

Set up two repos or a monorepo with clear **/frontend** and **/backend** separation — a monorepo is recommended for a hackathon-speed build.

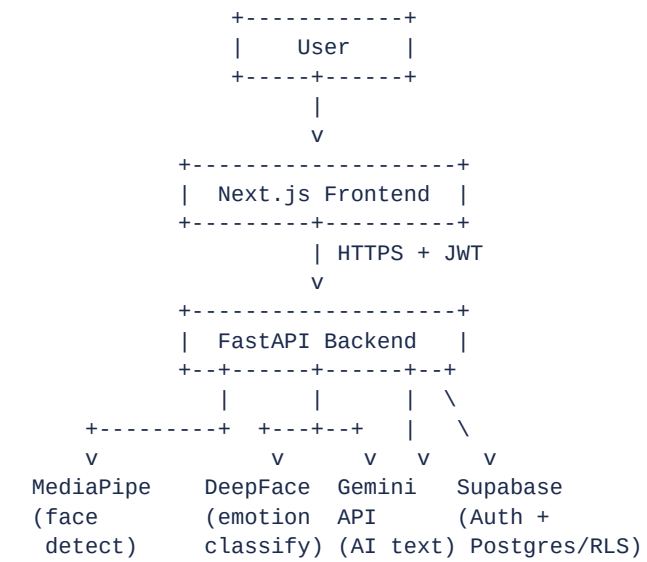
System Architecture

High-level request flow from user to backend services, and the authentication handshake that protects every API call.

Architecture Diagram — Mermaid

```
graph TD
    A[User] --> B[Next.js Frontend]
    B --> C[FastAPI Backend]
    C --> D[MediaPipe - Face Detection]
    C --> E[DeepFace - Emotion Classification]
    C --> F[Gemini API - Insight / Chat / Report Generation]
    C --> G[Supabase - Auth + Database]
    G --> H[(PostgreSQL + Row Level Security)]
```

Architecture Diagram — ASCII (PDF-safe)



Authentication Flow — Mermaid Sequence Diagram

```
sequenceDiagram
    participant U as User
    participant F as Frontend
    participant S as Supabase Auth
    participant B as Backend
    U->>F: Enter credentials
    F->>S: signIn(email, password)
    S-->>F: JWT session
    F->>B: Request + JWT (Authorization header)
    B->>S: Verify JWT
    S-->>B: Valid / user_id
    B-->>F: Protected response
```

Technology Stack Summary

Layer	Technology	Purpose
Frontend	Next.js (App Router) + TypeScript	UI, routing, server-side rendering

Layer	Technology	Purpose
Backend	FastAPI (Python)	API layer, orchestration of vision + AI calls
Database	Supabase (PostgreSQL)	Persistent storage with Row Level Security
Authentication	Supabase Auth	Email/password auth, JWT session issuance
AI	Gemini API	Insight, suggestion, chat, and report generation
Computer Vision	MediaPipe + DeepFace	Face detection and emotion classification
Charts	Recharts	Dashboard and report visualizations
Deployment	Vercel (frontend) / Render or Fly.io (backend)	Hosting and CI deploys
Styling	Tailwind CSS + shadcn/ui	Design system and component primitives
Animations	Framer Motion	Micro-interactions and page transitions
Language	TypeScript (frontend) / Python (backend)	Type safety and ML ecosystem access

1. Success Metrics

Measurable targets the build should be checked against, so “done” is objective rather than a feeling.

Metric	Target
Emotion detection response time	< 3 seconds from capture to result
AI insight generation time	< 5 seconds
Camera open success rate	Opens successfully on first permission grant, on Chrome/Edge/Safari desktop + mobile
End-to-end scan completion time	User can complete a full mood scan in < 30 seconds
Dashboard refresh	Updates immediately (no manual reload) after a new scan
Crisis-language fallback	Triggers on 100% of test crisis phrases, never a normal AI reply
Report generation	Weekly/monthly PDF available in < 8 seconds for a user with ≤ 30 days of data

2. Functional Requirements

Explicit, testable statements of what the system must do, grouped by area.

Auth

- **FR-01** — User can register with email and password.
- **FR-02** — User can log in and log out.
- **FR-03** — User can request a password reset and set a new password.
- **FR-04** — Protected routes redirect unauthenticated users to Login.

Scan / Emotion Detection

- **FR-05** — User can grant or deny camera permission from a consent screen.
- **FR-06** — User can capture a photo or retake it before submitting.
- **FR-07** — System detects a face in the captured image via MediaPipe.
- **FR-08** — System classifies emotion via DeepFace and computes a mood score.
- **FR-09** — System returns a friendly guidance message on no-face or low-confidence detection instead of a hard error.
- **FR-10** — System generates an AI-written insight explaining the detected mood.

Suggestions & Tasks

- **FR-11** — System selects 2–4 suggestions relevant to the detected emotion and time of day.
- **FR-12** — AI personalizes suggestion phrasing before display.
- **FR-13** — User can add a suggestion to “Today's Plan”, creating a task row.
- **FR-14** — User can toggle a task complete/incomplete.

Dashboard & Analytics

- **FR-15** — System stores every scan as mood history.
- **FR-16** — User can view mood history filtered by date range.
- **FR-17** — Dashboard shows current mood, mood score, latest AI insight, suggestions, and task checklist.
- **FR-18** — Dashboard renders daily timeline, weekly trend, emotion distribution, and task-completion charts.

Chat Companion

- **FR-19** — User can send a chat message and receive a mood-aware AI reply.
- **FR-20** — System injects a summary of the last 7 days of mood context into the chat prompt.
- **FR-21** — System pre-checks messages for crisis language and returns a safe fallback response, bypassing the normal model call, when detected.
- **FR-22** — Chat history persists and reloads on return to the page.

Reports

- **FR-23** — System generates a weekly and a monthly report summary via AI.
- **FR-24** — User can download a report as a PDF.
- **FR-25** — System shows an insufficient-data empty state when history is too short for a report.

Privacy & Settings

- **FR-26** — User can update personal info, theme, and notification preferences.
- **FR-27** — User can delete all mood history via a confirmation modal.
- **FR-28** — User can delete their account and all associated data via a confirmation modal.

3. Non-Functional Requirements

Category	Requirement
Performance	Scan pipeline and chat responses meet the Success Metrics targets under normal load.
Security	All traffic over HTTPS; secrets in environment variables, never in client bundles; RLS enforced on every ta
Privacy	Captured images are processed transiently and are not permanently stored unless the user explicitly opts i
Scalability	Stateless API layer so the backend can scale horizontally behind a load balancer post-hackathon.
Reliability	Graceful degradation: if the AI provider is unreachable, the app still returns emotion + mood score with a st
Accessibility	Keyboard-navigable forms, sufficient color contrast, alt text on informational icons, camera page usable wit
Responsiveness	Fully usable at mobile widths (360px+), with the Camera and Chat pages tested first since they are highest

4. Assumptions & Constraints

Assumptions

- User has a working webcam or a device camera.
- User has an active internet connection for the duration of a scan or chat session.
- The configured AI provider API is online and within rate limits.
- User grants camera permission when prompted (a denial path is still handled — see Error Handling).

Constraints

- Facial-emotion classification accuracy depends on lighting and camera quality; this is disclosed to the user, not silently corrected.
- The product is explicitly not intended for medical or psychiatric diagnosis — disclaimer is mandatory on every AI-insight screen.

- Requires a browser with camera (getUserMedia) support; no native camera API fallback in the MVP.
- Single-user accounts only — no multi-user/org accounts in scope.

5. Risks & Mitigation

Risk	Mitigation
Camera not available or permission denied	Offer an image-upload fallback so the flow isn't a dead end
AI API fails or times out	Return the deterministic emotion/mood score immediately; show a stubbed/fallback insigh
DeepFace confidence is low	Prompt the user to rescan with guidance (better lighting, face centered)
Slow or dropped internet connection	Show a loading state with timeout, then an inline retry action, no silent failure
No face detected	Friendly, specific guidance (e.g., “we couldn't find a face — try moving closer&rdqu
Crisis-language false negative in chat	Keep the pre-check pattern list conservative and err toward triggering fallback; test with k
Report generation timeout with large history	Cap the summarized window (last 7/30 days) before sending to the AI provider
Scope creep during hackathon	Enforce the Explicit Non-Goals list (Section 16) — no adaptive engine, no wearables, etc.

6. Error Handling Flows

Each of the following must resolve to a defined UI state — never a blank screen or unhandled exception.

Condition	Expected Behavior
No face detected	Show guidance message + retake option; do not call the AI provider
Multiple faces detected	Ask the user to ensure only one face is in frame; offer retake
Camera permission denied	Show the upload-image fallback; never dead-end the flow
AI provider timeout	Return partial result (emotion + mood score) with a stubbed insight; show a subtle retry-for-insigh
Backend unavailable (5xx / network error)	Show a toast/banner with retry; preserve any in-progress form state client-side
Database unavailable	Surface a generic “can't save right now” message; do not lose the captured scan re

7. State Management

Frontend state to model explicitly (e.g., in React context/hooks or a lightweight store):

- **Authentication state** — current user, session token, loading/hydration status.
- **Current mood** — latest scan result (emotion, mood score, confidence) available to Dashboard and Chat.
- **Scan status** — idle / requesting-permission / capturing / uploading / analyzing / result / error.
- **Chat history** — message list, streaming/typing indicator, mood-context badge state.
- **Dashboard data** — mood history range, chart data, loading and empty-state flags per widget.
- **Loading states** — per-request (scan, chat, report) rather than one global spinner, so unrelated UI stays interactive.
- **Error states** — per-request, mapped to the Error Handling table above, with user-facing copy attached.

8. AI Prompt Strategy

Each AI call should use a distinct, versioned prompt so outputs stay consistent and are easy to tune independently.

System Prompt (shared preamble)

```
You are Moodify's wellness assistant. You are not a medical
professional and never provide diagnosis or medical advice.
Keep responses concise, warm, and non-judgmental. Always be
compatible with a visible non-diagnostic disclaimer shown
alongside your output.
```

Mood Analysis Prompt (generate_insight)

```
Input: detected emotion label, confidence score, time of day.
Task: write a 2-3 sentence, first-person-friendly explanation
of what this mood snapshot might reflect, without diagnosing
or assuming causes outside the given data.
```

Suggestion Prompt (generate_suggestions)

```
Input: emotion label, mood score, time of day, base suggestion
text from the seeded suggestions table.
Task: rephrase 2-4 suggestions in a warm, personalized tone
without changing their underlying action.
```

Chat Prompt (POST /api/chat)

```
Input: user message, summarized mood context for the last 7
days, prior chat turns.
Task: respond in a mood-aware, supportive tone. If the
crisis-language pre-check flags the message, skip this prompt
entirely and return the fixed safe-fallback response.
```

Report Prompt (generate_report_summary)

```
Input: aggregated mood history for the selected period
(weekly or monthly), emotion distribution, task completion rate.
Task: produce a short narrative summary highlighting patterns,
without clinical language or predictive claims.
```

9. API Contracts

Each endpoint below documents request body, response body, and error responses, in addition to the route list in the TRD.

POST /api/scan

```
Request:
{
  "image": "base64..."
}

Response 200:
{
  "emotion": "Happy",
  "confidence": 94,
  "moodScore": 82,
  "insight": "..."
}

Response 422 (no face):
{ "error": "NO_FACE_DETECTED", "message": "..." }

Response 422 (low confidence):
{ "error": "LOW_CONFIDENCE", "message": "..." }
```

PATCH /api/tasks/{id}

```
Request:
{ "completed": true }

Response 200:
{ "id": "task_123", "completed": true, "updatedAt": "..." }

Response 404:
{ "error": "TASK_NOT_FOUND" }
```

GET /api/mood-history?from=&to;=

```
Response 200:
{
  "entries": [
    { "date": "2026-07-01", "emotion": "Calm", "moodScore": 70 }
  ]
}
```

POST /api/chat

```
Request:
{ "message": "..." }

Response 200:
{ "reply": "...", "moodContextUsed": true }

Response 200 (crisis fallback path):
{ "reply": "<fixed safe-fallback text>", "fallbackTriggered": true }
```

GET /api/reports/weekly | /api/reports/monthly

```
Response 200:
{ "reportId": "rep_123", "summary": "...", "period": "weekly" }
```

```
Response 422 (insufficient data):
{ "error": "INSUFFICIENT_DATA" }
```

GET /api/reports/{id}/pdf

Response 200: binary PDF stream. Response 404 if the report id does not exist or does not belong to the requesting user.

DELETE /api/user/mood-history · DELETE /api/user/account

```
Response 200:
{ "success": true }

Response 401: unauthenticated
Response 500: deletion failed, no partial state left behind
```

10. Database Relationships

High-level entity relationships (see Backend Schema doc for full column-level detail):

```
Users (1) ■■< MoodHistory (many)
Users (1) ■■< Tasks (many)
Users (1) ■■< ChatMessages (many)
MoodHistory (1) ■■< Tasks (many, via suggestion origin)
Suggestions (1) ■■< Tasks (many, template source)
Users (1) ■■< Reports (many)
```

*All child tables carry a **user_id** foreign key with Row Level Security so a user can only read or write their own rows.*

Detailed ER Diagram — Mermaid

```
erDiagram
    USERS ||--o{ MOOD_HISTORY : "has"
    USERS ||--o{ TASKS : "has"
    USERS ||--o{ CHAT_MESSAGES : "has"
    USERS ||--o{ REPORTS : "has"
    SUGGESTIONS ||--o{ TASKS : "templates"
    MOOD_HISTORY ||--o{ TASKS : "originates"

    USERS {
        uuid id PK
        string email
        string password_hash
        timestamp created_at
    }
    MOOD_HISTORY {
        uuid id PK
        uuid user_id FK
        string emotion
        int confidence
        int mood_score
        text ai_insight
        timestamp created_at
    }
    TASKS {
        uuid id PK
        uuid user_id FK
        uuid mood_history_id FK
        uuid suggestion_id FK
        string title
        bool completed
```

```
    timestamp created_at
}
SUGGESTIONS {
    uuid id PK
    string emotion
    string time_of_day
    text base_text
}
CHAT_MESSAGES {
    uuid id PK
    uuid user_id FK
    string role
    text content
    timestamp created_at
}
REPORTS {
    uuid id PK
    uuid user_id FK
    string period
    text summary
    timestamp created_at
}
```

Detailed ER Diagram — ASCII with Cardinality

```
USERS (1) --< MOOD_HISTORY (many)   [users.id = mood_history.user_id]
USERS (1) --< TASKS (many)          [users.id = tasks.user_id]
USERS (1) --< CHAT_MESSAGES (many)  [users.id = chat_messages.user_id]
USERS (1) --< REPORTS (many)        [users.id = reports.user_id]
SUGGESTIONS (1) --< TASKS (many)    [suggestions.id = tasks.suggestion_id]
MOOD_HISTORY (1) --< TASKS (many)   [mood_history.id = tasks.mood_history_id]
```

11. Project Folder Structure

```
moodify/
■■■ frontend/
■   ■■■ app/                # Next.js App Router pages
■   ■■■ components/
■   ■■■ lib/                # supabase client, api helpers
■   ■■■ hooks/
■   ■■■ styles/
■■■ backend/
■   ■■■ app/
■   ■   ■■■ api/             # FastAPI routers
■   ■   ■■■ services/        # vision pipeline, AI provider
■   ■   ■■■ models/
■   ■   ■■■ core/            # config, security, rate limiting
■   ■■■ tests/
■■■ docs/                   # PRD, TRD, this plan, schema, etc.
■■■ assets/
```

12. Coding Standards

Area	Convention
Files (frontend)	kebab-case for files, PascalCase for component names (e.g., scan-result.tsx exporting ScanResult)
Files (backend)	snake_case modules; one router per resource (scan.py, tasks.py, chat.py, reports.py)
API routes	Plural nouns, kebab-case where multi-word (/api/mood-history, /api/reports/weekly)
TypeScript	Strict mode on; no implicit any; shared types in a /types directory imported by both API calls and components
React	Functional components + hooks only; no class components; colocate a component's styles/tests with it
Python	Type hints on all function signatures; Pydantic models for every request/response body
Commits	Conventional commits (feat:, fix:, chore:) to keep the build log readable during a fast-moving hackathon

13. Security Design

- **Auth flow** — Supabase Auth issues JWTs; frontend stores the session via Supabase client, backend verifies the JWT on every protected request.
- **Password handling** — hashing is delegated entirely to Supabase Auth; the app never stores or logs raw passwords.
- **Row Level Security** — enabled on every table from day one (Phase 1), policies scoped to auth.uid() = user_id.
- **Transport security** — HTTPS enforced end-to-end; no mixed content.
- **Environment variables** — all keys (Supabase service role, AI provider key) live server-side only, never shipped to the client bundle; .env.example committed, .env gitignored.
- **API key protection** — the AI provider call is proxied through the backend; the frontend never holds a provider API key.
- **Rate limiting** — applied to /api/scan and /api/chat to control AI provider cost and abuse.

Browser Compatibility

Browser	Support Level
Chrome (desktop)	Full support — primary target for development and testing
Edge (Chromium)	Full support
Safari (desktop)	Full support; camera permission prompt styling differs from Chrome
Firefox	Full support; verify getUserMedia constraints during QA
Mobile Chrome (Android)	Full support — primary mobile target
Mobile Safari (iOS)	Supported; camera access requires HTTPS and must be triggered by a direct user gesture

Known limitations: iOS Safari does not allow camera auto-start on page load — the capture button must initiate the permission prompt. Older Firefox ESR builds may report a lower default camera resolution; the capture UI should not assume a fixed frame size.

14. UI Navigation Map


Landing
Login / Register
Dashboard
Scan --> Scan Result
Reports (Weekly / Monthly)
Chat
Profile / Settings

Complete Screen Inventory

Every screen in the application, with its purpose, main components, and the backend APIs it calls.

Landing

Purpose: Introduce Moodify to a first-time visitor and drive sign-up.

Main Components:

- Hero
- About
- Key Features
- How It Works
- Testimonials placeholder
- FAQ
- Footer

Backend APIs Used:

- None (static/marketing page)

Register

Purpose: Create a new account.

Main Components:

- Email/password form
- Validation messages
- Link to Login

Backend APIs Used:

- Supabase Auth: signUp

Login

Purpose: Authenticate an existing user.

Main Components:

- Email/password form
- Forgot Password link

Backend APIs Used:

- Supabase Auth: signInWithPassword

Forgot Password / Reset Password

Purpose: Recover account access.

Main Components:

- Email input
- Reset-link confirmation
- New password form

Backend APIs Used:

- Supabase Auth: resetPasswordForEmail, updateUser

Dashboard

Purpose: Display the emotional overview and day-to-day hub.

Main Components:

- Current Mood Card
- Mood Score
- AI Insight snippet
- Charts (timeline, trend, distribution, task completion)
- Task Checklist
- Suggestion Cards
- Chat Shortcut

Backend APIs Used:

- GET /api/mood-history
- GET /api/tasks
- PATCH /api/tasks/{id}

Scan — Consent & Capture

Purpose: Get camera consent and capture the photo used for emotion detection.

Main Components:

- Consent screen
- Camera preview
- Capture / Retake controls
- Upload-image fallback

Backend APIs Used:

- POST /api/scan

Scan Result

Purpose: Show the outcome of a scan.

Main Components:

- Emotion label
- Mood score
- AI insight text
- Non-diagnostic disclaimer
- Suggestion cards
- Add-to-plan action

Backend APIs Used:

- POST /api/scan (result payload)
- POST /api/tasks (on add-to-plan)

Chat

Purpose: Mood-aware conversational companion.

Main Components:

- Message list
- Input box
- Suggested prompt chips
- Mood-context badge

Backend APIs Used:

- GET /api/chat/history
- POST /api/chat

Reports

Purpose: Weekly/monthly narrative summary and PDF export.

Main Components:

- Weekly/Monthly toggle
- Summary text
- Supporting charts
- Download PDF button
- Insufficient-data empty state

Backend APIs Used:

- GET /api/reports/weekly
- GET /api/reports/monthly
- GET /api/reports/{id}/pdf

Profile / Settings

Purpose: Manage account, preferences, and data.

Main Components:

- Personal info form
- Theme toggle
- Notification prefs
- Delete Mood History (modal)
- Delete Account (modal)

Backend APIs Used:

- DELETE /api/user/mood-history
- DELETE /api/user/account

15. AI Ethics & Privacy

- User consent is required before camera access, shown on the /scan consent screen before the preview opens.
- Facial images are processed transiently for emotion classification and are not permanently stored unless the user explicitly opts in.
- Every AI-insight-bearing screen carries a visible, non-diagnostic disclaimer.
- Users can delete their mood history or their entire account at any time, with a confirmation modal on both actions.

16. Build Phases

Phase 1 — Foundations (Day 1, morning)

Goal: empty-but-running skeleton, auth working end-to-end.

- Scaffold Next.js (App Router, TypeScript, Tailwind, shadcn/ui installed).
- Scaffold FastAPI project with basic health-check endpoint.
- Create Supabase project: enable Auth (email/password), create all tables from Backend Schema doc §1, enable RLS policies as specified.
- Wire Supabase Auth into frontend: Register, Login, Forgot Password, Reset Password pages (App Flow §3).
- Add auth middleware/guards for protected routes.
- Set up environment variables (TRD §8–9) in both frontend and backend, with .env.example files committed.

Milestone check: *a user can register, log in, land on an empty Dashboard, and log out.*

Phase 2 — Core Scan Loop (Day 1, afternoon)

Goal: the single most important loop — camera → emotion → insight — works end-to-end, even with placeholder AI text.

- Build /scan page: consent screen, camera preview, capture/retake UI (App Flow §4).
- Build backend /api/scan endpoint: MediaPipe face detection → DeepFace emotion classification → mood score calculation (TRD §3).
- Handle NO_FACE_DETECTED and low-confidence paths (App Flow §5) before adding AI text generation — get the vision pipeline solid first.
- Integrate AI provider (AIProvider interface, TRD §4) for generate_insight only at this stage — suggestions can be stubbed.
- Build /scan/result page displaying emotion, mood score, AI insight, disclaimer.

Milestone check: *a real photo produces a real emotion label, mood score, and AI-written explanation, or a graceful no-face/low-confidence message.*

Phase 3 — Suggestions & Tasks (Day 1 evening / Day 2 morning)

- Seed the suggestions table with 25–30 entries (TRD §5) covering all 11 emotions and all times of day.
- Implement suggestion selection logic + generate_suggestions AI call to personalize phrasing.
- Wire “Add to Today's Plan” → creates wellness_tasks rows.
- Build task checklist UI (Dashboard-embedded), with complete/incomplete toggling (PATCH /api/tasks/{id}).

Milestone check: *after a scan, the user sees 2–4 suggestions, can add them as tasks, and can check them off.*

Phase 4 — Dashboard & Analytics (Day 2, midday)

- Build GET /api/mood-history with date-range filtering.
- Build Dashboard cards: current mood, mood score, AI insight snippet, suggestions, task checklist.
- Build charts (Recharts): daily timeline, weekly trend line, emotion distribution pie, task completion.
- Handle empty states for new users (App Flow §10).

Milestone check: *dashboard reflects real historical data and updates immediately after a new scan.*

Phase 5 — AI Chat Companion (Day 2, afternoon)

- Build chat_messages persistence + GET /api/chat/history.
- Implement POST /api/chat with mood-context injection (last 7 days summarized) and the crisis-language pre-check/fallback (TRD §4, App Flow §6).

- Build chat UI: message list, input, suggested prompt chips, mood-context badge (UI/UX Brief §4).

Milestone check: *chat responds in a mood-aware way; a test message containing crisis language reliably triggers the safe fallback path instead of a normal AI reply.*

Phase 6 — Reports (Day 2, evening)

- Implement generate_report_summary AI call + /api/reports/weekly and /api/reports/monthly endpoints.
- Build Reports page UI with weekly/monthly toggle and charts.
- Implement PDF generation (e.g., reportlab or weasyprint on the backend) and GET /api/reports/{id}/pdf.
- Handle insufficient-data empty state (App Flow §8).

Milestone check: *a user with at least a few days of scans can view and download a real weekly report PDF.*

Phase 7 — Privacy, Settings, Polish (Day 3, or compressed into Day 2 night)

- Build Profile/Settings page: personal info, theme toggle, notification prefs (can be inert/stubbed for MVP if time-constrained), Delete mood history, Delete account.
- Implement DELETE /api/user/mood-history and DELETE /api/user/account with confirmation modals (App Flow §9).
- Add rate limiting to /api/scan and /api/chat (TRD §7).
- Pass over disclaimer copy placement — confirm it appears on every AI-insight-bearing screen (PRD §8 non-functional requirement).
- Responsive pass, especially the Camera page on mobile widths (UI/UX Brief §10).
- Landing Page build (parallelizable, no backend dependency): Hero, About, Key Features, How It Works, Testimonials placeholder, FAQ, Footer.

Milestone check: *app is demo-ready — privacy controls work, no dead-end states, disclaimer consistently present.*

Suggested Build Order Rationale

The scan loop (Phase 2) is prioritized immediately after auth because it's the product's core differentiator and the riskiest technically (ML pipeline integration) — surfacing integration problems on Day 1 leaves time to fix them. Reports and Settings are pushed later because they're lower-risk, more standard CRUD/UI work that's less likely to blow up the timeline.

17. Explicit Non-Goals During Build

Do not spend build time on:

- Adaptive recommendation engine
- Mood prediction
- Wearable integration
- Therapist dashboard
- Multi-user / org accounts
- Voice / speech emotion analysis
- Multilingual support
- Native mobile app

All confirmed out of scope per PRD §7/§9.

18. Demo Scenario

A scripted walkthrough to rehearse before review, so the demo hits every core loop without dead air:

- 1. Log in.
- 2. Run a mood scan.
- 3. System detects and displays the mood.
- 4. AI explains the mood (disclaimer visible).
- 5. Add a suggested wellness task.
- 6. Mark the task complete.
- 7. Return to Dashboard — show it updated immediately.
- 8. Ask the chatbot a mood-related question.
- 9. Open the weekly report and download the PDF.

Demo Data Strategy

Live AI calls and a freshly-created account both introduce risk during judging — empty states and provider latency are not what the demo should be about. A seeded demo account removes that risk.

- **Demo account provisioning** — a seed script creates one or more pre-populated demo accounts (e.g., demo@moodify.app) via a backend admin task run before the review, not live in front of judges.
- **Seeded mood history** — roughly 14–21 days of scan records with varied emotions and mood scores, spread across different times of day, so charts show a real trend rather than a flat line.
- **Sample tasks** — a mix of completed and incomplete tasks tied to the seeded suggestions, so the task-completion chart has meaningful data on first load.
- **Sample reports** — at least one pre-generated weekly report (and a monthly one if the demo spans that far back) so the Reports page and PDF download work even if the AI provider is slow during the live demo.
- **Sample chat** — a short prior conversation already in chat_messages, so the mood-context badge and history rendering can be shown without waiting on a live model response first.
- **Why seeded data is necessary** — it guarantees every milestone in the Testing Checklist (Section 20) is demonstrable in seconds, keeps the demo resilient to a slow or rate-limited AI provider, and lets the live portion of the demo focus on the one or two things worth showing fresh (a real scan, a real chat reply).

19. Future Scope

Horizon	Ideas
Short term	Mood prediction, adaptive recommendations
Medium term	Voice emotion analysis, smartwatch integration
Long term	Therapist dashboard, enterprise wellness platform

20. Testing Checklist Before Demo/Review

- ■ Register → Login → Dashboard works
- ■ Camera consent denial doesn't dead-end the user
- ■ A real scan produces emotion + mood score + AI insight + disclaimer
- ■ No-face and low-confidence cases handled gracefully
- ■ Suggestions → Tasks → completion toggling works and reflects in analytics
- ■ Charts render correctly with real data and with empty/sparse data
- ■ Chat responds mood-aware; crisis fallback verified with a test phrase
- ■ Weekly report generates and PDF downloads
- ■ Delete mood history and delete account both work and are confirmed via modal
- ■ Every AI-insight screen shows the non-diagnostic disclaimer